

Data Exploration Q&A: High-Cost Medicare Beneficiary Classifier

1. What does one row represent?

One row in the gold modeling table represents one Medicare beneficiary in one calendar year. The precise unit of analysis is a beneficiary-year, identified by the composite key `bene_id` + `year`.

This means the gold table is not organized at the claim, visit, prescription, transaction, or service-line level. Instead, raw claim and event records are aggregated into annual beneficiary-level features. For example, one row for beneficiary 1000001 in 2008 contains that beneficiary's 2008 demographics, enrollment information, chronic condition burden, utilization counts, provider counts, claim days, costs by claim type, and total annual claim cost.

The raw source tables start at different grains:

Source table Raw row represents
Beneficiary summary One beneficiary-year
Inpatient claims One inpatient claim
Outpatient claims One outpatient claim
Carrier claims One carrier claim record with line-level fields
Prescription drug events One prescription drug event

The gold transformation resolves these different source grains by grouping claims by `bene_id` and `year` before joining them to the beneficiary-year base table.

2. How many rows are in the data?

The local extracted bronze CSV data contains 11,494,963 data rows, excluding header rows.

Source	Data rows
Beneficiary summary	343,644
Carrier claims	4,741,335
Inpatient claims	66,773
Outpatient claims	790,790
Prescription drug events	5,552,421
Total	11,494,963

This count describes the raw extracted data. The gold modeling table is smaller because it aggregates multiple claim and prescription records into one row per beneficiary-year.

3. How many columns are in the raw data?

The extracted source files have different schemas and column counts.

Source	Columns
Beneficiary summary	32
Carrier claims	142
Inpatient claims	81
Outpatient claims	76
Prescription drug events	8

Carrier claims has the widest raw schema because it contains repeated line-level fields within each claim record. The gold table narrows and standardizes these source schemas into a curated modeling feature set.

4. Is fresher data available for this problem?

The best interpretation is that this project demonstrates a reusable pipeline. If newer restricted claims data became available, the same bronze-to-silver-to-gold design could be adapted and rerun.

5. What percentage of the most important column is missing?

The most important derived column for this project is `annual_claim_cost`, because it defines the high-cost target. In the gold table, this field is built from typed cost components and missing claim aggregates are filled with zero before calculating annual cost.

The raw cost fields underneath the target were checked directly. The direct cost fields for inpatient, outpatient, and prescription drug events had 0.00% missing values in the extracted files:

Raw cost field	Rows checked	Missing values	Null rate
Inpatient CLM_PMT_AMT	66,773	0	0.00%
Outpatient CLM_PMT_AMT	790,790	0	0.00%
Prescription TOT_RX_CST_AMT	5,552,421	0	0.00%

Carrier payment is constructed from repeated line payment fields rather than one single raw payment column. The silver transformation sums `LINE_NCH_PMT_AMT_1` through `LINE_NCH_PMT_AMT_13`, treating missing line values as zero. This is appropriate because not every claim uses all 13 line slots.

6. Are there duplicate rows? How would we know?

	Check		Result	
	Beneficiary-year rows		343,644	
	Distinct bene_id + year keys		343,644	
	Duplicate bene_id + year keys		0	
	Extra duplicate rows		0	

The Databricks SQL check for the gold table is:

```
SELECT bene_id, year, COUNT(*) AS row_count
FROM gold_beneficiary_year_features
GROUP BY bene_id, year
HAVING COUNT(*) > 1;
```

If this query returns no rows, the gold table has no duplicate primary keys. The pipeline also writes a duplicate_bene_year_count metric into the gold_audit_summary table.

7. Are the columns the right data types?

The raw CSV files are string-like at ingestion, but the silver layer explicitly casts important columns into appropriate types before they are used for modeling.

	Field type		Example columns		Silver type	
	Dates		BENE_BIRTH_DT, CLM_FROM_DT, CLM_THRU_DT, SRVC_DT		date	
	Integers		SEGMENT, CLM_UTLZTN_DAY_CNT, DAYS_SUPLY_NUM		int	
	Costs/payments		CLM_PMT_AMT, TOT_RX_CST_AMT, carrier line payments		double	
	Flags		Chronic condition indicators		boolean	
	IDs/codes		DESYNPUF_ID, CLM_ID, provider IDs, diagnosis codes		string	

This matters because the target variable depends on summing numeric cost fields. If payment amounts were accidentally kept as strings, the model could silently receive incorrect features. In the gold table, key fields such as annual_claim_cost, log costs, rates, and cost shares are numeric.

8. Have key feature distributions been inspected?

Yes. The key cost fields are not bell-curved. They are right-skewed, with many low-cost

Feature	Median	90th pct	99th pct	Max	Shape
Inpatient payment	7,000	19,000	57,000	57,000	Right-skewed
Outpatient payment	80	700	3,300	3,300	Strongly right-skewed
Prescription drug cost	20	160	550	570	Right-skewed

Additional distribution checks found:

Feature	Finding
Inpatient payment	3.23% zero values and 0.08% negative values
Outpatient payment	3.81% zero values and 0.32% negative values
Prescription drug cost	9.86% zero values and 0.00% negative values

The chronic-condition burden distribution was also inspected after correcting the CMS flag coding. Using 1 as present and 2 as not present, the distribution is:

Chronic condition count	Share
0	36.66%
1	12.89%
2	12.03%
3	10.73%
4	9.09%
5 or more	18.60%

This revealed an important pipeline issue: the original silver parser only treated Y as true, but the CMS chronic-condition flags use 1/2 coding. That would have incorrectly made chronic-condition counts zero. The silver parser was fixed to treat 1, Y, YES, TRUE, and T as true values.

9. Who or what might be underrepresented?

Underrepresented slice	Reason
Real Medicare beneficiaries	The dataset is synthetic, so it mimics structure but not exact real population patterns.
Recent patients	The data covers 2008-2010 and misses modern care patterns.
Medicare Advantage beneficiaries	The data is closer to fee-for-service Medicare claims.
Younger populations	Medicare data does not generalize to most commercially insured adults, children, or uninsured patients.
Social determinants of health	Claims do not include detailed income, housing, food access, transportation, or caregiver support.
Clinical severity	Claims lack full notes, lab values, vitals, symptoms, and patient-reported outcomes.
Real intervention histories	The data does not contain care-management actions and responses.

These gaps mean the model should be interpreted as a claims-based risk prototype, not a complete picture of patient health or care need.

10. What time period does the data cover?

The data covers calendar years 2008, 2009, and 2010.

Source	Time period
Beneficiary summary	2008, 2009, 2010
Inpatient claims	2008-2010
Outpatient claims	2008-2010
Carrier claims	2008-2010
Prescription drug events	2008-2010

The data is outside the window of COVID-19, modern telehealth expansion, recent prescription drug pricing, current Medicare Advantage growth, newer coding patterns, and recent value-based care programs. This limits direct generalization to today's Medicare environment.

11. Is the target variable balanced?

No. The target variable is intentionally imbalanced. High cost status is defined as being

Class	Meaning	Expected share
0	Not high cost	About 90%
1	High cost	About 10%

This means a naive model could achieve about 90% accuracy by always predicting the majority class. For this reason, accuracy alone is not enough. The project evaluates models with ROC-AUC, precision-recall AUC, recall, top-k capture, and lift.

12. Could any features be leaking the answer?

Yes. Leakage is a real risk if same-year cost and utilization features are used to predict same-year high-cost status. The high-cost target is derived from annual claim cost, so any feature that directly or indirectly contains same-year cost information can encode the answer.

Potential leakage feature	Why risky
inpatient_total_cost	Component of annual claim cost
outpatient_total_cost	Component of annual claim cost
carrier_total_cost	Component of annual claim cost
rx_total_cost	Component of annual claim cost
annual_cost_log1p	Direct transformation of annual claim cost
cost_per_enrollment_month	Derived from annual claim cost
Cost shares	Derived from same-year cost components
Same-year claim counts and claim days	May not exist before the prediction period ends

The defensible prospective design is to use year t features to predict high-cost status in year $t + 1$. For example, 2008 features should predict 2009 high-cost status, and 2009 features should predict 2010 high-cost status. If same-year data is used for demonstration, it should be described as risk segmentation rather than true prospective prediction.

13. What happens with unexpected user input?

The app has some protection, but more validation would make it stronger. FastAPI and

	Input issue		Current risk	
	Negative costs		Could produce unstable or unrealistic predictions if not blocked.	
	Negative claim counts		Should be rejected as invalid.	
	Zero enrollment months		Could break rate calculations if rates are derived from user input.	
	Unknown categories		May fail preprocessing or encode unpredictably.	
	Extremely large values		Could produce out-of-distribution scores.	
	Missing required fields		Rejected by the API schema.	

A production-ready version should reject negative costs and counts, restrict enrollment months to 1-12, map unknown categories to an explicit unknown bucket or reject them, and cap or warn on extreme values using training-percentile bounds.

14. Would rerunning the pipeline produce the same gold table?

For the raw-to-gold transformation, the answer should mostly be yes if the same raw files and code are used. The row count, primary keys, column names, and column types should be reproducible because the transformations are deterministic.

However, several things could shift:

	Possible shift		Explanation	
	chronic_condition_count		This intentionally changes after the flag-parser fix.	
	Audit timestamps		processed_at_utc changes on every run.	
	Physical row order		Spark and Delta do not guarantee row order unless explicitly ordered.	
	Tiny floating-point differences		Distributed aggregation can theoretically change floating-point order.	
	Model artifacts		Training can shift if splits, seeds, package versions, or hyperparameters are not fixed.	

The most important current reproducibility note is that rerunning silver and gold after the flag-parser fix will produce a better but different gold table. That is intentional because the old chronic-condition values were wrong.

15. What step should be strengthened most?

The silver-layer data validation and source-code normalization step should be strengthened

This is the layer where raw CMS-specific encodings become typed, model-ready fields. The chronic-condition issue is a good example: the pipeline ran successfully, but because the parser expected Y/N while the source used 1/2, the resulting chronic-condition counts were wrong. That kind of silent error can distort every downstream model.

The silver layer should be improved with:

Improvement	Purpose
Allowed-value checks	Catch unexpected encodings such as 1/2 vs Y/N.
Null-rate checks	Detect missingness spikes before gold aggregation.
Schema contracts	Confirm expected columns and types exist.
Distribution checks	Catch impossible all-zero or constant features.
Duplicate-key checks	Protect the bene_id + year grain.
Fail-fast rules	Stop the pipeline when critical features are invalid.

The silver layer is the contract between messy raw CMS files and every downstream feature, model, and app result. Strengthening it is the highest-impact pipeline improvement.

Summary
